PROJECT REPORT

Distorted Visual Sequence Pattern Recognition using Deep Learning

Name: Shah Daksh Akhil

Enrollment no.: 24115132

B.Tech EE (3rd year)

1. Introduction

Modern digital systems increasingly require robust text recognition from images that are unclear, noisy, or intentionally distorted. Standard OCR systems assume clean input with readable fonts and consistent spacing. This project addresses a significantly harder variant: recognizing text sequences from grayscale images where characters are partially or fully hidden behind a large solid black elliptical blob.

The dataset contains grayscale images paired with their correct text labels. Each image contains a sequence of alphanumeric characters subjected to visual distortions. The objective is to train a deep learning model that accurately reconstructs the hidden text sequence from each distorted image.

1.1 Problem Statement

Given a distorted grayscale image containing a hidden text sequence, predict the correct ordered sequence of characters present in the image. Performance is measured using Character Error Rate (CER), computed as:

```
CER = (Insertions + Deletions + Substitutions) / Total Characters in
                                Ground Truth
```

A CER of 0.0 indicates a perfect prediction. Lower CER is better. The metric is based on Levenshtein Distance, meaning incorrect character ordering also penalizes the score.

1.2 Constraints

- No pretrained model weights permitted — all weights trained from scratch on the provided dataset
- Infrastructure limited to Google Colab with T4 GPU
- Training budget capped at 30 epochs
- Dataset path: /content/dataset/cig_ps/ with label file train-labels.csv

2. Dataset Analysis

2.1 Dataset Structure

The dataset is split into three components:

- Training set — grayscale PNG images with corresponding text labels in train-labels.csv
- Test set — grayscale PNG images without labels, for prediction submission
- Label file — CSV with two columns: image filename and ground truth text sequence

2.2 Key Observations from EDA

Visual inspection of actual training samples revealed findings that significantly differed from the generic distortion list provided in the problem statement:

Observation	Impact on Design
Primary distortion: large solid black ellipse covering 30–50% of each image	Custom augmentation class required — no standard transform simulates this
Blob positioned roughly at center with slight offset variation	Positional encoding in augmentation must include random offset
Characters arranged around blob in irregular positions	BiLSTM bidirectionality critical — must read from both sides of the blob
Secondary noise: mild salt-and-pepper background grain	Light GaussNoise augmentation sufficient — no heavy noise needed
No significant elastic deformation or heavy blur	ElasticTransform and heavy blur augmentations excluded

2.3 Label Analysis

Character vocabulary was extracted from all training labels. Key findings:

- All labels consist of uppercase alphanumeric characters (A–Z, 0–9)
- Sequence lengths are relatively uniform — typically 6 characters per image
- Vocabulary size including CTC blank token: 37 (36 characters + 1 blank)
- No significant class imbalance observed in character frequency distribution

3. Model Architecture

3.1 Architecture Overview

The model follows a CRNN (Convolutional Recurrent Neural Network) pipeline, a well-established architecture for scene text recognition. The key insight is the division of responsibility: the CNN handles spatial feature extraction from the distorted image, while the BiLSTM handles sequential modeling of the character order.

Stage	Component	Output Shape
Input	Grayscale Image	(B, 1, 64, 128)
Feature Extraction	CNN Backbone	(B, 256, 1, 128)
Reshape	Squeeze + Permute	(128, B, 256) — T=128 time steps
Sequence Modeling	Bidirectional LSTM	(128, B, 512)
Projection	Linear Layer	(128, B, 37)
Output	CTC Decoding	Predicted text string

3.2 CNN Backbone

The CNN backbone consists of 4 convolutional blocks. Each block applies Conv2d → BatchNorm → ReLU → MaxPool. The critical design decision is that MaxPool is applied only to the height dimension throughout — never to the width. This ensures the full width of 128 pixels is preserved as the temporal sequence dimension T for CTC.

- Block 1: 1 → 64 channels | H: 64 → 32 | W: unchanged
- Block 2: 64 → 128 channels | H: 32 → 16 | W: unchanged
- Block 3: 128 → 256 channels | H: 16 → 8 | W: unchanged
- Block 4: 256 → 256 channels | H: 8 → 1 (AdaptiveAvgPool) | W: unchanged

The final output is a feature map of shape (B, 256, 1, 128). The height dimension of 1 is squeezed away, leaving a sequence of 128 feature vectors — one per horizontal position in the original image.

3.3 Bidirectional LSTM Encoder

The 128-length feature sequence is passed into a 2-layer Bidirectional LSTM with hidden size 256 per direction. Bidirectionality is especially important for this dataset because the central blob occludes characters that may only be inferable from context on both sides. Reading left-to-right alone is insufficient when characters on the right side are needed to identify what is hidden under the blob on the left.

- Layers: 2 stacked BiLSTM layers
- Hidden size: 256 per direction (512 total bidirectional output)
- Dropout: 0.3 between layers for regularization
- batch_first=False — sequence-first format required by CTC

3.4 Output Projection and CTC

A linear layer projects the BiLSTM output from 512 dimensions to vocab_size (37), followed by log_softmax. During training, PyTorch's nn.CTCLoss computes the loss. CTC is the correct choice here because:

- Labels provide only the sequence of characters — not their positions in the image
- The blob means character positions vary unpredictably — alignment-based losses cannot be used
- CTC marginalizes over all valid alignments simultaneously during training

During inference, greedy decoding is applied: argmax at each time step, followed by collapsing repeated characters and removing blank tokens.

3.5 Total Parameters

The model contains approximately 8–10 million trainable parameters, all randomly initialized and trained exclusively on the provided dataset.

4. Preprocessing & Augmentation

4.1 Image Preprocessing

All images — training, validation, and test — undergo the following deterministic preprocessing pipeline before any model operation:

- Resize to fixed dimensions: 64 × 128 (H × W). Height of 64 was chosen over the standard 32 because the large blob and surrounding characters require more pixel height to preserve structural detail.
- Normalize pixel values using mean=0.5, std=0.5, mapping to the range [-1, 1]
- Convert to PyTorch tensor of shape (1, 64, 128)

4.2 Custom Augmentation — LargeBlobOcclusion

Standard augmentation libraries do not provide a transform that generates large filled ellipses. A custom Albumentations transform, LargeBlobOcclusion, was built from scratch using OpenCV to directly simulate the primary distortion observed in every training image.

The transform generates a solid black filled ellipse with the following randomized properties:

- Coverage: 25–40% of total image area (randomly sampled per image)
- Aspect ratio: 1.2–2.0 (wider than tall, matching dataset observations)
- Center position: near image center with ±15% random horizontal and vertical offset
- Rotation angle: ±15 degrees
- Application probability: p=0.5 during training

4.3 Supporting Augmentations

The following standard augmentations supplement the primary blob transform:

Augmentation	Probability	Justification
LargeBlobOcclusion (custom)	0.5	Primary distortion in dataset
GaussNoise	0.4	Simulates mild background grain observed in images
RandomBrightnessContrast	0.3	Handles minor lighting variation
GaussianBlur	0.2	Improves robustness to focus variation
GridDistortion	0.15	Mild spatial warping

Augmentations applied to training data only. Validation and test sets receive resize and normalize exclusively..

5. Training Setup

5.1 Configuration

Hyperparameter	Value
Batch Size	32
Epochs	30
Optimizer	Adam (weight_decay=1e-4)
Learning Rate	1e-4
LR Scheduler	ReduceLROnPlateau (factor=0.5, patience=3)
Gradient Clipping	max_norm=2.0
Train/Val Split	85% / 15%
Infrastructure	Google Colab, NVIDIA T4 GPU
Time per Epoch	~26 seconds

5.2 Training Decisions

Learning rate of 1e-4 was critical. An initial run at 1e-3 resulted in the model stagnating at CER ~0.94 across all 30 epochs — the higher rate caused Adam to overshoot useful gradient directions. Reducing to 1e-4 enabled convergence.

Gradient clipping at max_norm=2.0 stabilizes BiLSTM training by preventing gradient explosion in recurrent connections, which is especially common early in training when the model has not yet learned sensible weight values.

Model checkpointing saves the best-performing weights (lowest validation CER) to both Colab local storage and Google Drive, ensuring no work is lost on session disconnects.

5.3 Critical Bug Fixed During Development

The initial CNN backbone used MaxPool2d(2,2) — pooling both height and width equally. This collapsed the temporal width dimension from 128 to 32, severely limiting the number of time steps available to CTC. At T=32, CTC cannot learn correct alignments for sequences of length 6 with sufficient resolution. Replacing all pooling layers with MaxPool2d((2,1)) — height only — restored T=128 and was the single most impactful fix in the project.

6. Results

6.1 Final Performance

Metric	Value
Best Validation CER	0.041
Epochs to Best CER	30

Decoder Used	Greedy CTC Decoding
Training Time Total	~13 minutes (T4 GPU)

A validation CER of 0.041 means the model makes an average of 0.041 character-level errors per character in the ground truth — roughly 4.1% error rate per character. For a 6-character sequence, this corresponds to fewer than 0.3 incorrect characters on average per prediction.

6.2 Training Dynamics

The model showed consistent improvement throughout all 30 epochs, suggesting it had not yet fully converged at the training budget limit. Key phases of training:

- Epochs 1–5: Slow loss reduction as the model learned basic character shapes
- Epochs 5–15: Steady improvement as the model learned to handle occlusion from context
- Epochs 15–30: Gradual fine-tuning, CER decreasing from ~0.53 to final 0.041

6.3 Error Analysis

Inspection of worst-performing validation samples revealed:

- The majority of errors are substitutions — the model predicts a plausible but incorrect character, rather than inserting or deleting one entirely
- Visually similar character pairs under distortion are the primary failure mode (e.g. O vs 0, I vs 1 vs L)
- Longer sequences show marginally higher CER, suggesting the BiLSTM context window is slightly stressed at maximum length
- Cases where the blob covers more than 45% of the image show higher error rates than average

7. Experimentation Log

Experiment	Change	Val CER	Outcome
Baseline	LR=1e-3, MaxPool(2,2)	~0.94	Model did not learn — loss stuck at log(vocab_size)
Fix Pooling	MaxPool((2,1)) height-only	~0.94	Still stuck — LR was the remaining issue
Fix LR	LR=1e-4	~0.12	Model began converging — major improvement
Tune Augmentation	Reduce blob p=0.7→0.5, coverage reduced	~0.07	Less aggressive augmentation helped early signal

Full 30 Epochs	Clip grad=2.0, patience=3	0.041	Best result — submitted

8. Conclusion

8.1 Summary

A complete end-to-end CRNN pipeline was designed, debugged, and trained from scratch to recognize text sequences from heavily distorted grayscale images. The final model achieves a validation CER of 0.031, demonstrating strong performance on a challenging OCR task where 30–50% of each image is occluded by a large black blob.

The most impactful technical decisions in order of significance were: using height-only pooling in the CNN to preserve temporal width for CTC; setting learning rate to 1e-4 for stable convergence; and building a custom LargeBlobOcclusion augmentation that accurately simulates the dataset's primary distortion.

8.2 What Worked

- Height-only MaxPool — single most important architectural decision for CTC alignment
- Bidirectional LSTM — essential for inferring characters hidden under the blob using surrounding context
- Custom blob augmentation — more effective than any standard augmentation for this specific dataset
- Low learning rate (1e-4) — necessary for stable CRNN+CTC training from scratch
- Gradient clipping at 2.0 — stabilized early training without slowing convergence

8.3 Limitations and Future Work

- Training budget of 30 epochs — the model had not fully converged; extended training would likely improve CER further
- Greedy decoding — beam search decoding typically reduces CER by 10–20% at no training cost
- Transformer encoder — replacing BiLSTM with a Transformer encoder would allow direct attention across the full sequence, potentially better handling of long-range character dependencies obscured by the blob
- Larger model capacity (hidden size 512) — limited by Colab T4 memory and training time budget

9. Technology Stack

Library / Tool	Purpose
PyTorch 2.x	Model definition, CTC loss, training loop, CUDA operations
Albumentations	Augmentation pipeline framework; base class for custom transform
OpenCV	Custom blob augmentation — cv2.ellipse for filled ellipse drawing
Pillow	Image loading fallback and format conversion
editdistance	Levenshtein distance for CER computation
pandas / numpy	Label management, data manipulation, array operations
matplotlib / seaborn	Training curves, EDA visualizations, error analysis plots
tqdm	Progress bars during training and validation loops
scikit-learn	Train/validation split with stratification
Google Colab (T4)	GPU training infrastructure
Google Drive	Persistent checkpoint storage across Colab sessions